Lab 4

Name: Vankadara Sri Kruthi

SRN: PES2UG24CS574

Subject: Software Engineering

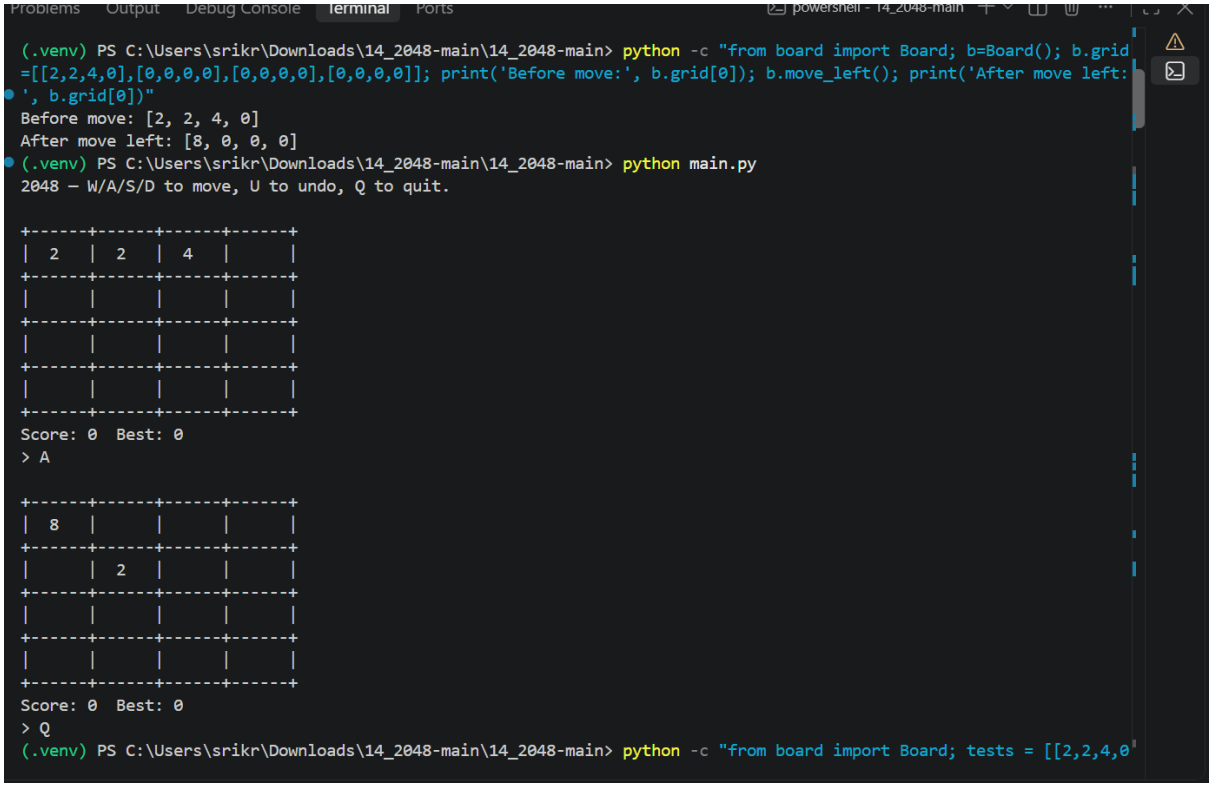
Section : 5J

Github link :

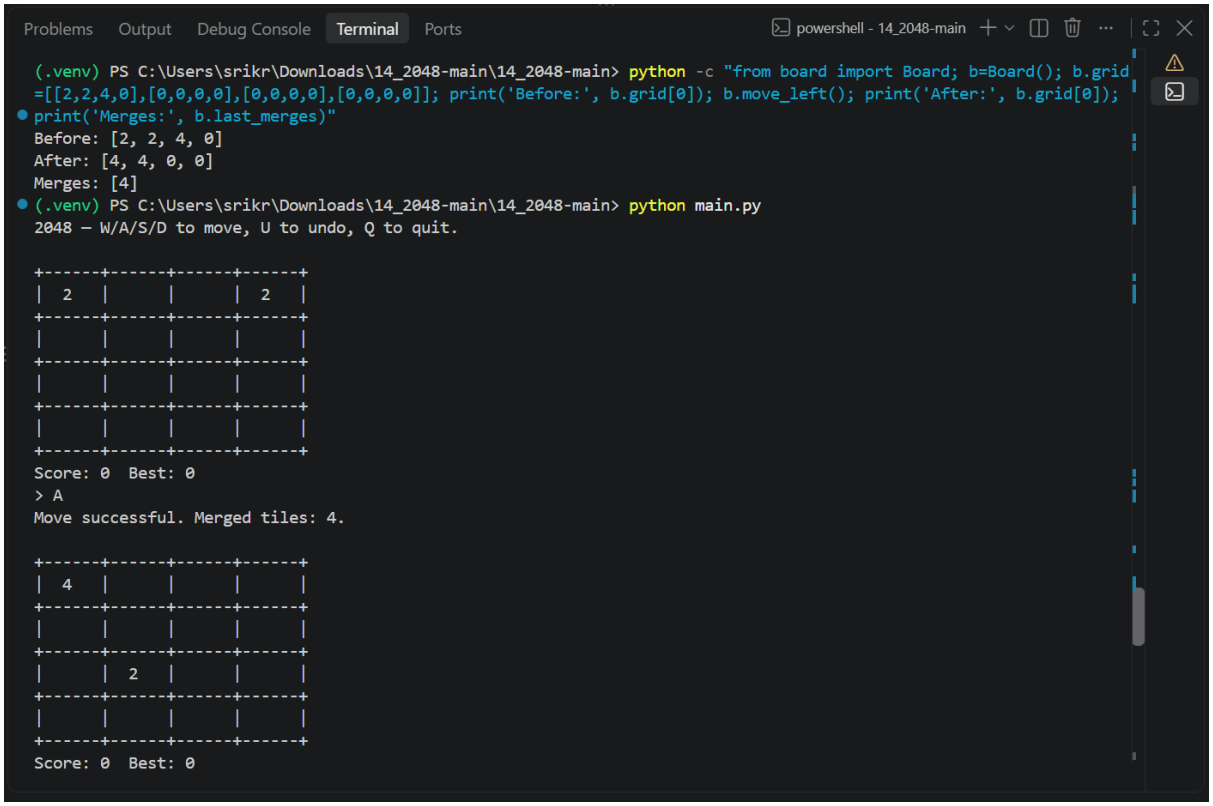
https://github.com/kruthivankadara-dev/14_2048

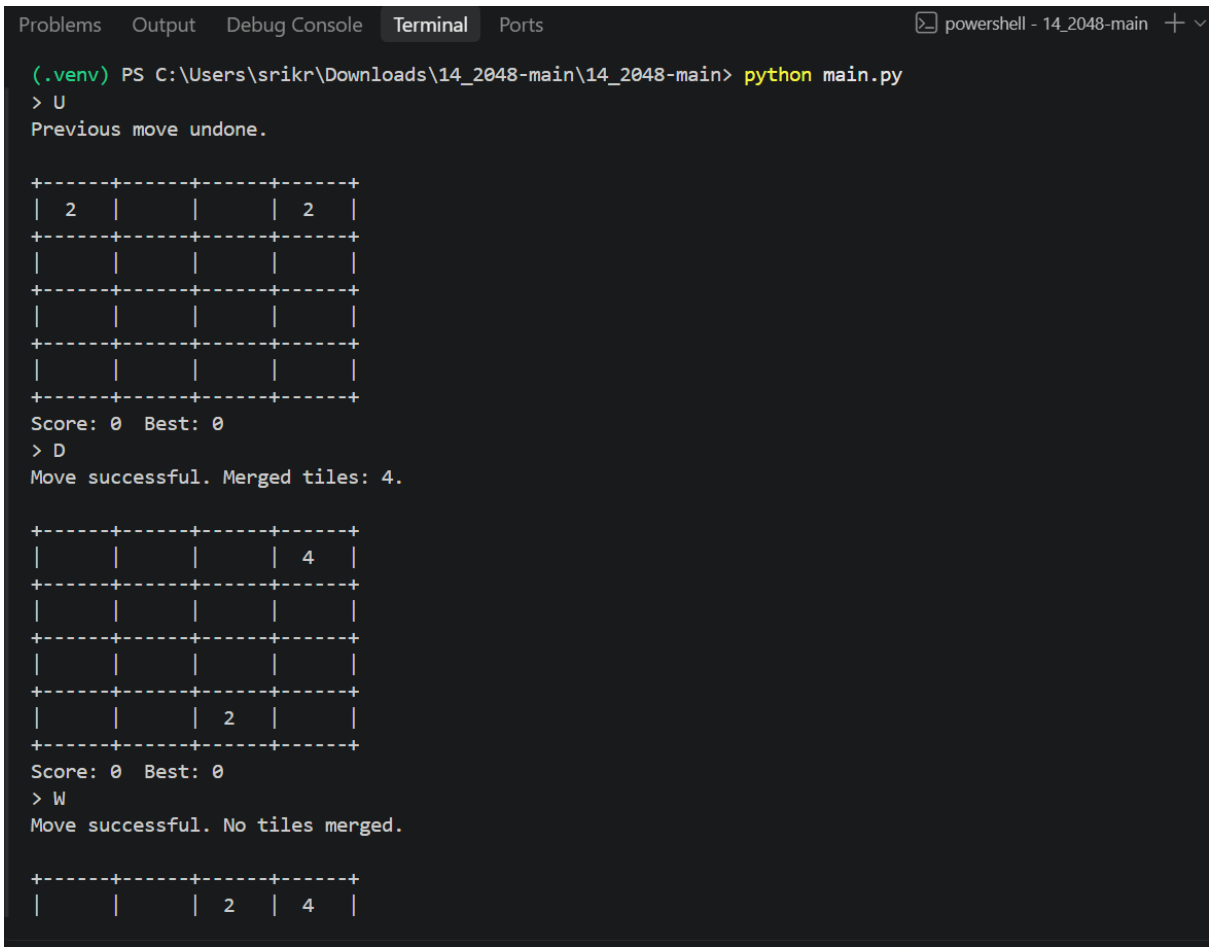
Screenshots :

BEFORE :



AFTER:


Problems Output Debug Console Terminal Ports
powershell - 14_2048-main
(.venv) PS C:\Users\srikr\Downloads\14_2048-main\14_2048-main> python -c "from board import Board; b=Board(); b.grid =[[2,2,4,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; print('Before:', b.grid[0]); b.move_left(); print('After:', b.grid[0]); print('Merges:', b.last_merges)"
Before: [2, 2, 4, 0]
After: [4, 4, 0, 0]
Merges: [4]
(.venv) PS C:\Users\srikr\Downloads\14_2048-main\14_2048-main> python main.py
2048 — W/A/S/D to move, U to undo, Q to quit.
| 2 | | | 2 |
Score: 0 Best: 0
> A
Move successful. Merged tiles: 4.
| 4 | | | |
| | 2 | | |
Score: 0 Best: 0


Problems Output Debug Console Terminal Ports
powershell - 14_2048-main
(.venv) PS C:\Users\srikr\Downloads\14_2048-main\14_2048-main> python main.py
> U
Previous move undone.
| 2 | | | 2 |
Score: 0 Best: 0
> D
Move successful. Merged tiles: 4.
| 4 |
| 2 |
Score: 0 Best: 0
> W
Move successful. No tiles merged.
| 2 | 4 |

UPDATED CODE :

board.py:

```
import random

SIZE = 4


class Board:
    def __init__(self):
        self.grid = [[0] * SIZE for _ in range(SIZE)]
        self.score = 0
        self.add_random_tile()
        self.add_random_tile()

        self.last_merges = []

    def add_random_tile(self):
        empty = [(r, c) for r in range(SIZE) for c in range(SIZE) if
self.grid[r][c] == 0]
        if empty:
            r, c = random.choice(empty)
            self.grid[r][c] = 4 if random.random() < 0.1 else 2


    @staticmethod

    @staticmethod
    def slide_line_with_merges(line):
        values = [x for x in line if x]
        result = []
        merges = []
        i = 0

        while i < len(values):
            if i + 1 < len(values) and values[i] == values[i + 1]:
                merged_value = values[i] * 2
                result.append(merged_value)
                merges.append(merged_value)
                i += 2
            else:
                result.append(values[i])
                i += 1

        result += [0] * (SIZE - len(result))
        return result, merges

    @staticmethod
```

```
    def slide_line(line):
        result, _ = Board.slide_line_with_merges(line)
        return result




    def move_left(self):
        changed = False
        self.last_merges = []

        for r in range(SIZE):
            old = self.grid[r][:]
            new, merges = self.slide_line_with_merges(old)
            self.grid[r] = new
            self.last_merges.extend(merges)
            changed |= old != new

        return changed

    def move_right(self):
        changed = False
        self.last_merges = []

        for r in range(SIZE):
            old = self.grid[r][:]
            new, merges = self.slide_line_with_merges(list(reversed(old)))
            new = list(reversed(new))
            self.grid[r] = new
            self.last_merges.extend(merges)
            changed |= old != new

        return changed

    def move_up(self):
        changed = False
        self.last_merges = []

        for c in range(SIZE):
            old = [self.grid[r][c] for r in range(SIZE)]
            new, merges = self.slide_line_with_merges(old)

            for r in range(SIZE):
                self.grid[r][c] = new[r]

            self.last_merges.extend(merges)
            changed |= old != new

        return changed
```

```
    def move_down(self):
        changed = False
        self.last_merges = []

        for c in range(SIZE):
            old = [self.grid[r][c] for r in range(SIZE)]
            new, merges = self.slide_line_with_merges(list(reversed(old)))
            new = list(reversed(new))

            for r in range(SIZE):
                self.grid[r][c] = new[r]

            self.last_merges.extend(merges)
            changed |= old != new

        return changed


    def can_move(self):
        if any(0 in row for row in self.grid):
            return True
        for r in range(SIZE):
            for c in range(SIZE):
                if c + 1 < SIZE and self.grid[r][c] == self.grid[r][c + 1]:
                    return True
                if r + 1 < SIZE and self.grid[r][c] == self.grid[r + 1][c]:
                    return True
        return False
```

game.py

```
from board import Board


class Game:
    def __init__(self):
        self.board = Board()
        self.best_score = 0
        self.history = None  # Stores one previous successful move

    def display(self):
        print("\n" + "+------+------+------+------+")
        for row in self.board.grid:
            print("|" + "|".join(
                f"{x:^6}" if x else f"{' ':^6}" for x in row
```

```
            ) + "|")
            print("+------+------+------+------+")
        print("Score:", self.board.score, " Best:", self.best_score)

    def has_won(self):
        return any(2048 in row for row in self.board.grid)

    def move(self, key):
        moves = {
            "a": self.board.move_left,
            "d": self.board.move_right,
            "w": self.board.move_up,
            "s": self.board.move_down
        }

        if key not in moves:
            return False

        # Save the current state before attempting the move.
        previous_grid = [row[:] for row in self.board.grid]
        previous_score = self.board.score

        changed = moves[key]()

        if not changed:
            # An unchanged move must not replace undo history.
            return False

        # Replace the previous snapshot with this successful move's
        # starting state. This supports exactly one level of undo.
        self.history = (previous_grid, previous_score)

        # Update the best score for this session.
        self.best_score = max(self.best_score, self.board.score)

        # Spawn a tile only after a successful move, unless the game
        # has already been won.
        if not self.has_won():
            self.board.add_random_tile()

        return True

    def undo(self):
        if self.history is None:
            return False

        previous_grid, previous_score = self.history
```

```
        # Restore the board and score without generating a tile.
        self.board.grid = [row[:] for row in previous_grid]
        self.board.score = previous_score

        # Clear history so the same move cannot be undone repeatedly.
        self.history = None

        return True

    def run(self):
        print("2048 — W/A/S/D to move, U to undo, Q to quit.")

        while True:
            self.display()

            if self.has_won():
                print("You reached 2048! You win!")
                return

            if not self.board.can_move():
                print("Game over! No legal moves remain.")
                return

            key = input("> ").strip().lower()

            if key == "q":
                print("Game ended.")
                return

            if key == "u":
                if self.undo():
                    print("Previous move undone.")
                else:
                    print("Nothing to undo.")
                continue

            if key not in ("w", "a", "s", "d"):
                print("Invalid command. Use W/A/S/D, U, or Q.")
                continue


            if self.move(key):
                merges = self.board.last_merges

                if merges:
                    merged_values = ", ".join(map(str, merges))
                    print(
                        f"Move successful. Merged tiles: {merged_values}."
```

```
                )
            else:
                print("Move successful. No tiles merged.")
        else:
            print("No change. Try another direction.")
```

CHAT HISTORY:

Chatgpt link : https://chatgpt.com/c/6ad094db-bc00-83ec-b704-a6755447a338

Please inspect the existing `game.py` and `board.py` files and implement the missing game-state handling while preserving the existing architecture.

Requirements:

1. Detect when a tile reaches 2048 and end the game with an appropriate message.
2. Detect when the board is full and no legal moves remain, and end the game.
3. Add a new random tile only after a successful move that changes the board.
4. Do not add a tile after an invalid command or an unchanged move.
5. Preserve the existing movement behaviour and scoring rules.
6. Keep all game state in memory. Do not introduce CSV, JSON, SQLite, or any other persistent storage.

Make the smallest necessary changes to the existing implementation. Explain how each requirement is satisfied and provide test cases for reaching 2048, a full board with no legal moves, and an unchanged move.

Task 2 — Game-State Handling and Tile Creation

We will make the smallest necessary changes to your existing `game.py` and `board.py`, preserving the current project structure and terminal controls.

The implementation will handle three important situations:

- Win detection: End the game when a tile reaches 2048.
- Game over: End the game when the board is full and no legal moves remain.
- Correct tile creation: Spawn a random tile only after a valid move actually changes the board.

We will keep all state in memory and introduce no additional dependencies or persistent storage.

1. Update `game.py`

Open `game.py` and replace its contents with the following version. It preserves the existing `Game` class, `Board` class, and W/A/S/D, U, and Q commands.

```
from game import Game
```

Correction: use the complete implementation below in `game.py` (the import at the top should be `from board import Board`).

```
from board import Boardclass Game:    def __init__(self):        self.board
= Board()        self.best_score = 0        self.history = []    def
display(self):        print("\n" + "+------+------+------+------+")
for row in self.board.grid:            print("|" + "|".join(
f"{x:^6}" if x else f"{' ':^6}" for x in row            ) + "|")
print("+------+------+------+------+")        print("Score:",
self.board.score, " Best:", self.best_score)    def has_won(self):
return any(2048 in row for row in self.board.grid)    def move(self, key):
moves = {            "a": self.board.move_left,            "d":
self.board.move_right,            "w": self.board.move_up,            "s":
self.board.move_down        }        if key not in moves:            return
False        changed = moves[key]()        # Spawn a tile only after a
successful move.
```

Use only the complete `game.py` implementation above. The tiny `from game import Game` snippet before it is not part of the code to paste.

2. Do you need to change `board.py`?

Your existing `board.py` already has a `can_move()` method. Keep it; it checks for empty cells or adjacent equal tiles.

Make sure the corrected `slide_line()` method from Task 1 is still present. No additional changes to `board.py` are needed for Task 2.

3. Test the requirements

Run these tests in the VS Code terminal from the inner project directory.

Test A — Reaching 2048

Run:

```
python -c "from game import Game; g=Game();
g.board.grid=[[1024,1024,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; g.move('a');
print('Win detected:', g.has_won()); print('First row:', g.board.grid[0])"
```

Expected result:

```
Win detected: True
First row: [2048, 0, 0, 0]
```

Test B — Full board with no legal moves

Run:

```
python -c "from board import Board; b=Board();
b.grid=[[2,4,2,4],[4,2,4,2],[2,4,2,4],[4,2,4,2]]; print('Game over:', not
b.can_move())"
```

Expected result:

```
Game over: True
```

Test C — Unchanged move does not spawn a tile

Run:

```
python -c "from game import Game; g=Game();
g.board.grid=[[2,0,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; before=sum(x!=0 for
row in g.board.grid for x in row); changed=g.move('a'); after=sum(x!=0 for
row in g.board.grid for x in row); print('Move changed:', changed);
print('Tiles before:', before); print('Tiles after:', after)"
```

Expected result:

```
Move changed: False
Tiles before: 1
Tiles after: 1
```

Test D — Invalid command

Run the game:

```
python main.py
```

Type `x` and press Enter. The game should display an invalid-command message and should not spawn a tile.

4. What has been implemented?

Requirement	Implementation
Detect 2048	`has_won()`
Detect no legal moves	Existing `Board.can_move()`
Spawn only after a changed move	`if changed` condition
Avoid spawning after reaching 2048	`not self.has_won()` condition
Invalid commands do not spawn tiles	Input validation before movement
No persistent storage	All state remains in memory

Once these tests pass, Task 2 is complete. Save both files before moving to Task 3.

Extend the existing 2048 game to implement one-level undo functionality and best-score tracking.

Requirements:

1. Before each successful move, save the previous board state and score.
2. Allow the user to undo the most recent successful move using the existing `U` command.
3. Undo must restore both the board contents and the score exactly as they were before that move.
4. Undo must not generate a new random tile.
5. Support only one level of undo; a new successful move should replace the previous undo state.
6. Invalid commands and unchanged moves must not overwrite the undo state.
7. Track the best score achieved during the current game session.
8. Preserve the existing terminal interface and avoid persistent storage.

Inspect the existing `Game` and `Board` classes before making changes. Keep the implementation modular, explain the changes, and provide tests covering undo after a successful merge, score restoration, repeated undo, and undo after an unchanged move.

Task 3 — Undo Functionality and Best-Score Tracking

We will update your existing `Game` class in `game.py`. Your `Board` class can remain unchanged, provided it already contains the corrected merge logic from Task 1 and the movement and scoring behavior from Task 2.

The implementation will maintain one undo snapshot in memory. A snapshot is saved only when a move actually changes the board. It includes both the grid and the score, and undo restores them without generating a random tile.

The best score will remain tracked for the current game session, even if you undo a move.

1. Update `game.py`

Replace the complete contents of `game.py` with the following code. It preserves your existing terminal interface and controls while adding undo and best-score tracking.

```
from board import Boardclass Game:    def __init__(self):        self.board
= Board()        self.best_score = 0        self.history = None  # Stores
one previous successful move    def display(self):        print("\n" + "+--
----+------+------+------+")        for row in self.board.grid:
print("|" + "|".join(                f"{x:^6}" if x else f"{' ':^6}" for x
in row            ) + "|")            print("+------+------+------+------
+")        print("Score:", self.board.score, " Best:", self.best_score)
def has_won(self):        return any(2048 in row for row in
self.board.grid)    def move(self, key):        moves = {            "a":
self.board.move_left,            "d": self.board.move_right,
```

```
"w": self.board.move_up,            "s": self.board.move_down        }
if key not in moves:            return False        # Save the current
state before attempting the move.        previous_grid = [row[:] for row in
self.board.grid]        previous_score = self.board.score
```

Save the file using `Ctrl + S`.

2. How the implementation works

Requirement	Implementation
Save state before a successful move	Copies the grid and score before attempting movement; stores the snapshot only if the board changes.
Undo with `U`	Calls `self.undo()` from the existing game loop.
Restore board and score	Restores both from the saved snapshot.
No random tile during undo	`undo()` only restores in-memory state.
One-level undo	A successful move replaces the previous snapshot; undo clears it.
Unchanged moves preserve history	Returns before changing `self.history`.
Best score for the session	Updated after successful moves and not reduced by undo.
No persistent storage	Uses ordinary Python objects in memory.

3. Test the implementation

Run these tests from the inner project directory in your VS Code terminal.

Test A — Undo after a successful merge

```
python -c "from game import Game; g=Game();
g.board.grid=[[2,2,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; g.board.score=0;
g.move('a'); print('After move:', g.board.grid[0]); print('Undo
available:', g.undo()); print('Restored row:', g.board.grid[0])"
```

Expected: the first row is `[4, 0, 0, 0]` after moving left, and undo restores `[2, 2, 0, 0]`.

A random tile may be spawned after the successful move. Undo should remove that tile by restoring the original board.

Test B — Score restoration

```
python -c "from game import Game; g=Game();
g.board.grid=[[2,2,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; g.board.score=10;
g.move('a'); g.board.score=14; print('Undo:', g.undo()); print('Restored
score:', g.board.score); print('Best score:', g.best_score)"
```

Expected restored score: `10`.

The best score should not decrease when the previous state is restored.

Test C — Repeated undo

```
python -c "from game import Game; g=Game();
g.board.grid=[[2,2,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; g.move('a');
print('First undo:', g.undo()); print('Second undo:', g.undo())"
```

Expected:

```
First undo: True
Second undo: False
```

Test D — Unchanged move preserves undo history

```
python -c "from game import Game; g=Game();
g.board.grid=[[2,0,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; g.move('a');
print('Move left unchanged; history:', g.history)"
```

For this test, the board must remain unchanged when moving left. Since it contains only one tile already at the left edge, the expected result is `False` for the move, and no new undo snapshot should be created.

For a stronger test of preserving existing history, first make a successful move, then attempt an unchanged move in another direction, and finally undo. Undo should still restore the state from before the successful move.

4. Final interactive test

Run:

```
python main.py
```

Test these controls:

- `A`, `D`, `W`, `S` — move tiles.
- `U` — undo the most recent successful move.
- `U` again — should report that there is nothing to undo.
- `Q` — quit the game.

Task 3 is complete once these tests pass. Note that the original `Board` methods you shared do not update the score on merges. This implementation preserves the existing scoring behavior and restores whatever score the board had before the move. If scoring is expected to increase on merges, that should be implemented and tested separately without accidentally changing the scope of this task.

Improve the existing terminal-based 2048 game by adding concise feedback for successful moves and tile merges.

Requirements:

1. Display appropriate feedback when a move successfully changes the board.

2. Provide merge feedback when tiles combine, including the resulting tile value where practical.
3. Generate feedback at the game-action level rather than inside internal board-scanning or movement helper functions.
4. Do not display success or merge feedback for invalid commands or moves that leave the board unchanged.
5. Ensure that a single accepted move produces one action-level result, even if multiple merges occur.
6. Preserve the existing W/A/S/D controls, U for undo, and Q for quitting.
7. Avoid unnecessary dependencies and keep the implementation modular.

Inspect the existing code before modifying it. Explain how feedback is generated and provide tests for successful moves, multiple merges in one move, unchanged moves, and invalid commands.

Task 4 — Move and Merge Feedback

We will update your existing `game.py` and make a small change to `board.py` only if needed. The goal is to provide feedback once per successful move, with additional details about any merges, without printing misleading messages for invalid or unchanged moves.

Your existing architecture already separates responsibilities:

- `board.py` handles tile movement and merging.
- `game.py` handles user commands and the game loop.

We'll preserve that separation. The board will return information about merges, and the game will display the feedback after the move is completed. We won't print messages from internal movement helpers.

1. Update `board.py`

Keep your existing `Board` class. Make the following changes.

A. Add merge tracking in `__init__`

At the end of `__init__`, after the two calls to `self.add_random_tile()`, add:

```
self.last_merges = []
```

B. Replace `slide_line()` with these two methods

This preserves the corrected merge algorithm from Task 1 and returns the values created by merges.

```
    @staticmethod    def slide_line_with_merges(line):        values = [x
for x in line if x]        result = []        merges = []        i = 0
while i < len(values):            if i + 1 < len(values) and values[i] ==
values[i + 1]:                merged_value = values[i] * 2
```

```
result.append(merged_value)                merges.append(merged_value)
i += 2            else:                result.append(values[i])
i += 1        result += [0] * (SIZE - len(result))        return result,
merges    @staticmethod    def slide_line(line):        result, _ =
Board.slide_line_with_merges(line)        return result
```

C. Update the four movement methods

Replace your existing `move_left()`, `move_right()`, `move_up()`, and `move_down()` methods with the versions below. They collect merge information without printing anything.

```
    def move_left(self):        changed = False        self.last_merges =
[]        for r in range(SIZE):            old = self.grid[r][:]
new, merges = self.slide_line_with_merges(old)            self.grid[r] =
new            self.last_merges.extend(merges)            changed |= old !=
new        return changed    def move_right(self):        changed = False
self.last_merges = []        for r in range(SIZE):            old =
self.grid[r][:]            new, merges =
self.slide_line_with_merges(list(reversed(old)))            new =
list(reversed(new))            self.grid[r] = new
self.last_merges.extend(merges)            changed |= old != new
return changed    def move_up(self):        changed = False
self.last_merges = []        for c in range(SIZE):            old =
[self.grid[r][c] for r in range(SIZE)]            new, merges =
self.slide_line_with_merges(old)
```

2. Update `game.py`

Keep your existing `Game` class and Task 2–3 functionality. We only need to update the successful-move feedback in `run()`.

Find this section at the end of your `run()` method:

```
            if self.move(key):                print("Move successful.")
else:                print("No change. Try another direction.")
```

Replace it with:

```
            if self.move(key):                merges =
self.board.last_merges                if merges:
merged_values = ", ".join(map(str, merges))                    print(
f"Move successful. Merged tiles: {merged_values}."                )
else:                    print("Move successful. No tiles merged.")
else:                print("No change. Try another direction.")
```

This produces exactly one action-level success message. If multiple merges happen during one move, their resulting values are included in that same message.

For example:

```
Move successful. Merged tiles: 4, 4.
```

If a move changes the board but no tiles merge:

```
Move successful. No tiles merged.
```

If the move leaves the board unchanged:

```
No change. Try another direction.
```

Invalid commands continue to display the invalid-command message already implemented in Task 2.

3. Test the feedback

Save both files with `Ctrl + S`.

Test A — Successful move with a merge

Run this command:

```
python -c "from board import Board; b=Board();
b.grid=[[2,2,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; changed=b.move_left();
print('Changed:', changed); print('Merges:', b.last_merges)"
```

Expected:

```
Changed: True
Merges: [4]
```

Test B — Multiple merges in one move

```
python -c "from board import Board; b=Board();
b.grid=[[2,2,2,2],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; changed=b.move_left();
print('Changed:', changed); print('Merges:', b.last_merges); print('Row:',
b.grid[0])"
```

Expected:

```
Changed: True
Merges: [4, 4]
Row: [4, 4, 0, 0]
```

Test C — Unchanged move

```
python -c "from board import Board; b=Board();
b.grid=[[2,0,0,0],[0,0,0,0],[0,0,0,0],[0,0,0,0]]; changed=b.move_left();
print('Changed:', changed); print('Merges:', b.last_merges)"
```

Expected:

```
Changed: False
Merges: []
```

Test D — Invalid command

Run:

```
python main.py
```

Enter `x` and press Enter. The game should display an invalid-command message without displaying success or merge feedback.

4. Final verification

Check that:

- A successful move produces one action-level success message.
- Merge values are included when merges occur.
- Multiple merges are reported together.
- Unchanged moves do not claim success.
- Invalid commands do not produce success or merge feedback.
- Undo, best-score tracking, win detection, and game-over detection still work.